FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Franco Gerard C. Isip
Section:	AN43
Professor:	Ms. Crisola G. Tan
Date:	October 14, 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

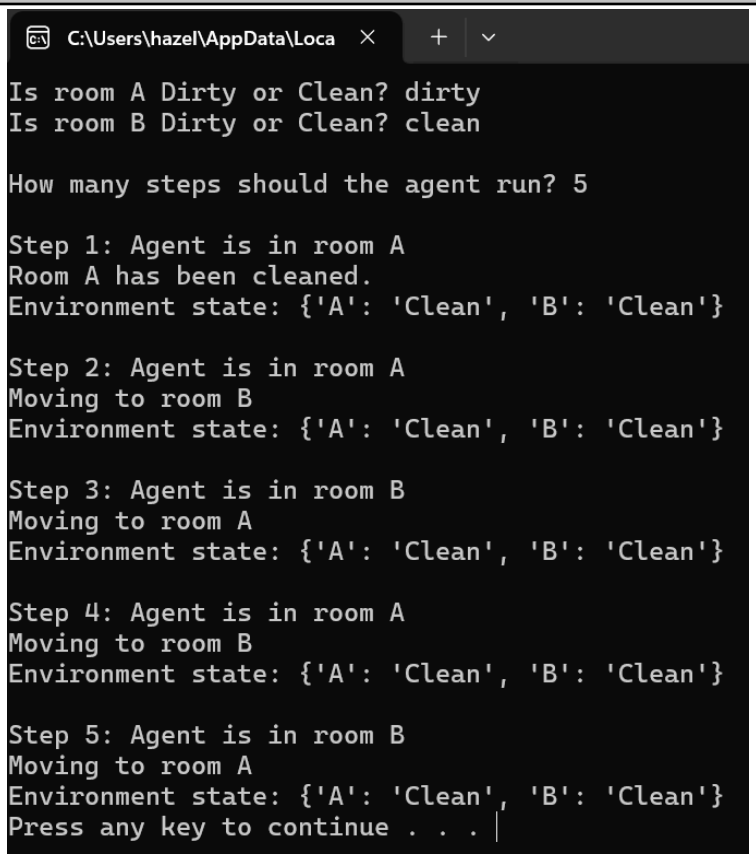
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

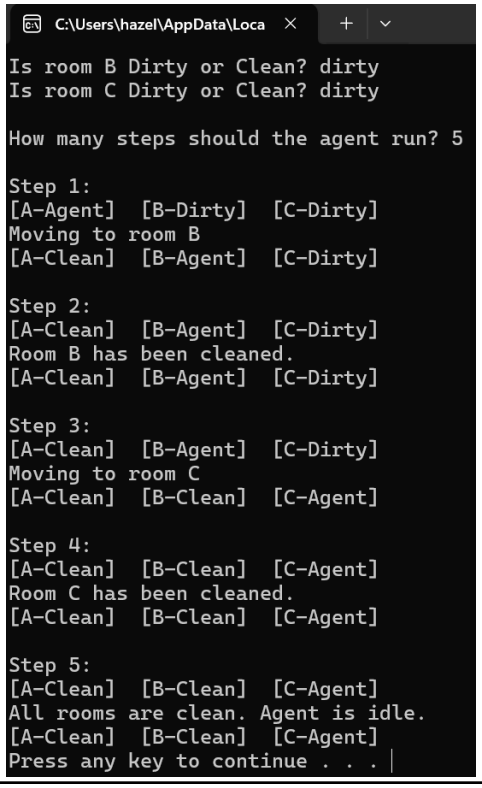
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
import random


# ========================================
# TASK 1: SET UP THE ENVIRONMENT
```

```
# ========================================

environment = {
    "A": "",
    "B": "",
    "C": ""
}

for room in environment:
    while True:
        state = input(
            f"Is room {room} Dirty or Clean? "
        ).strip().capitalize()

        if state in ("Dirty", "Clean"):
            environment[room] = state
            break
        else:
            print("Invalid input. Enter Dirty or Clean.")


def is_dirty(environment, room):
    return environment[room] == "Dirty"


def clean_room(environment, room):
    environment[room] = "Clean"


# ========================================
# BONUS: DISPLAY THE ROOM GRID
# ========================================

def display_grid(environment, agent_location):
    rooms = []

    for room in environment:
```

```
        if room == agent_location:
            rooms.append(f"[{room}-Agent]")
        else:
            rooms.append(f"[{room}-{environment[room]}]")

    print(" ".join(rooms))


# ========================================
# TASK 2: IMPLEMENT THE AGENT
# ========================================

class Agent:
    def __init__(self):
        self.location = "A"
        self.idle = False

    def move(self, destination):
        self.location = destination

    def perceive_and_act(self, environment):

        # If every room is clean, become idle
        if all(
            state == "Clean"
            for state in environment.values()
        ):
            self.idle = True
            return "All rooms are clean. Agent is idle."

        # Rule 1: Clean the current room if dirty
        if is_dirty(environment, self.location):
            clean_room(environment, self.location)
            return (
                f"Room {self.location} has been cleaned."
            )
```

```
        # Rule 2: Randomly select a dirty room
        dirty_rooms = [
            room for room in environment
            if is_dirty(environment, room)
        ]

        destination = random.choice(dirty_rooms)
        self.move(destination)

        return f"Moving to room {destination}"


# =============================================
# TASK 3: RUN THE SIMULATION
# =============================================

agent = Agent()

while True:
    try:
        steps = int(input(
            "\nHow many steps should the agent run? "
        ))

        if steps >= 0:
            break
        else:
            print("Enter a non-negative number.")
    except ValueError:
        print("Enter a valid whole number.")

for step in range(1, steps + 1):
    print(f"\nStep {step}:")

    # Display the grid before the action
    display_grid(environment, agent.location)
```

```
    # Perform the action
    action = agent.perceive_and_act(environment)
    print(action)

    # Display the updated grid
    display_grid(environment, agent.location)

    # Stop if the agent is idle
    if agent.idle:
        break

input("\nPress any key to continue...")
```

Sample Run:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\VSCode Programs> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "c:\VSCode Programs\.venv-1\Scr
(.venv-1) PS C:\VSCode Programs> & "c:\VSCode Programs\.venv-1\Scripts\python.exe" "c:/VSCode Programs/Python/TA2_Isip.py"
Is room A Dirty or Clean? Clean
Is room B Dirty or Clean? Dirty
Is room C Dirty or Clean? Dirty

How many steps should the agent run? 8

Step 1:
[A-Agent] [B-Dirty] [C-Dirty]
Moving to room C
[A-Clean] [B-Dirty] [C-Agent]

Step 2:
[A-Clean] [B-Dirty] [C-Agent]
Room C has been cleaned.
[A-Clean] [B-Dirty] [C-Agent]

Step 3:
[A-Clean] [B-Dirty] [C-Agent]
Moving to room B
[A-Clean] [B-Agent] [C-Clean]

Step 4:
[A-Clean] [B-Agent] [C-Clean]
Room B has been cleaned.
[A-Clean] [B-Agent] [C-Clean]

Step 5:
[A-Clean] [B-Agent] [C-Clean]
All rooms are clean. Agent is idle.
[A-Clean] [B-Agent] [C-Clean]

Press any key to continue...
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. The agent follows two main rules: if all rooms are clean, it becomes idle; otherwise, if its current room is dirty, it cleans it. If its current room is clean but another room is dirty, it randomly chooses a dirty room and moves there.
2. It checks whether every room is clean first. If not, it checks the room where it is currently located: a dirty room is cleaned, while a clean room causes it to move to a randomly chosen dirty room. Moving and cleaning happen on separate steps.
3. If all rooms are already clean when the simulation starts, then on the first step the agent reports that all rooms are clean and becomes idle. In this program, that means all three rooms, A, B, and C, must be clean. If the step count is zero, the agent does not run and so does not check the rooms.

```
Press any key to continue...
(.venv-1) PS C:\VSCode Programs> & "c:\VSCode Programs\.venv-1\Scripts\python.exe" "c:/VSCode Programs/Python/TA2_Isip.py"
Is room A Dirty or Clean? Clean
Is room B Dirty or Clean? Clean
Is room C Dirty or Clean? Clean

How many steps should the agent run? 4

Step 1:
[A-Agent] [B-Clean] [C-Clean]
All rooms are clean. Agent is idle.
[A-Agent] [B-Clean] [C-Clean]

Press any key to continue...
```

4. If I needed the agent to handle more than two rooms, I would add each new room to the environment dictionary and set the agent's starting location to one of those rooms. I used loops over the dictionary, so the program would automatically include the added rooms when asking for their states, finding dirty rooms, and displaying the grid.